[Mandic and Chambers, 2001, Chung et al., 2014].

3.1.1 *Gated Recurrent Units (GRU)*

A Gated Recurrent Unit (GRU) is a type of Recurrent Neural Network (RNN) designed to capture long-term dependencies in sequential data by addressing the vanishing gradient problem common in traditional RNNs. GRUs use two gates—update and reset—to manage information flow. The update gate controls how much past information is passed along to future states, while the reset gate determines how much past information is forgotten, allowing GRUs to retain relevant information over longer sequences.

The following set of equations defines a GRU unit:

$$\begin{aligned} z &= \sigma(x_t u^z + s_{t-1} w^z + b^z), \\ r &= \sigma(x_t u^r + s_{t-1} w^r + b^r), \\ v &= \tanh(x_t u^v + (s_{t-1} \times r) w^v + b^v), \\ s_t &= z \times v + (1 - z) s_{t-1}, \end{aligned} \tag{3.1}$$

where u^z, w^z and b^z are learned parameters governing the *update gate* z, while u^r, w^r and b^r are the learned parameters for the *reset gate* r. The candidate activation v determined by the input x_t and the previous output s_{t-1}, and is influenced by the learned parameters: u^v, w^v and b^v. Finally, the output s_t is a combination of the candidate activation v and the previous state s_{t-1} controlled by the *update gate* z. Figure 3.1 depicts an illustration of a GRU unit.

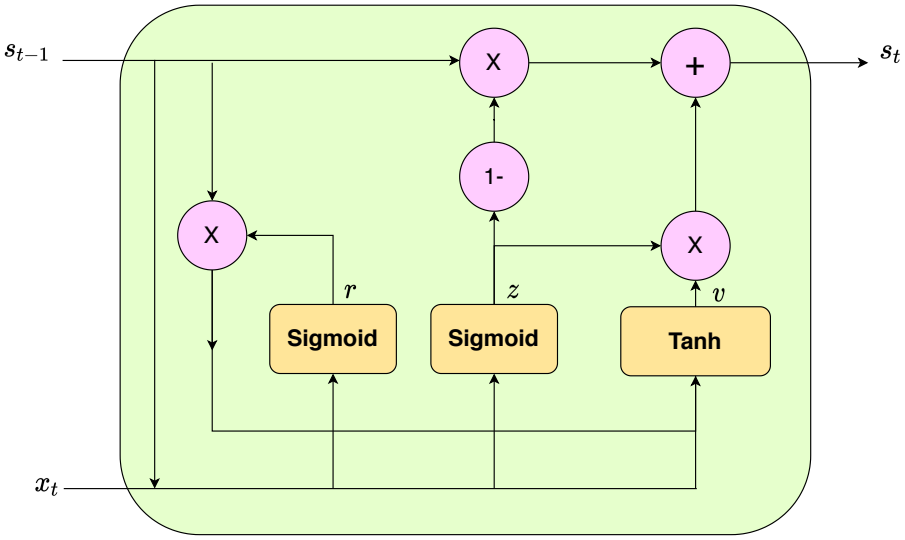
GRUs form the foundational unit of the HRNN model, detailed in Section 3.1.2 as well as our BiHRNN model detailed in Section 3.2.

3.1.2 *HRNN*

Next, we proceed with a description of the HRNN model. The following notations apply to both HRNN and Bidirectional HRNN models:

Let $\mathcal{I} = \{n\}_{n=1}^{N}$ represent dataset graph nodes, each associated with a parent π_n. For node n, x_t^n is its observed value at time t, and X_t^n represents the sequence up to t. A parametric function g (a GRU node) predicts the next value in the sequence, learning parameters θ_n to predict x_{t+1}^n. Assuming Gaussian errors, the likelihood of the time series is modeled as a product of normal distributions, with τ_n^{-1} as the error variance. A hierarchical informative prior connects each node's parameters to its parent's, with the prior $p(\theta_n | \theta_{\pi_n}, \tau_{\theta_n})$ using τ_{θ_n} as a precision parameter. Higher τ_{θ_n} values indicate stronger parameter connections between node n and its parent π_n. Instead of globally optimizing τ_{θ_n}, the HRNN sets $\tau_{\theta_n} = e^{\alpha + C_n}$, where α is a hyperparameter and C_n is the Pearson correlation between node n and its parent π_n. This ensures that node n stays close to π_n in parameter space, especially when correlation is high. For the root node, a

Figure 3.1. *An illustration of a GRU unit.*



Each line represents a vector, connecting the output of one node to the inputs of others. Pink circles indicate point-wise operations, while yellow boxes represent learned neural network layers. When lines merge, it signifies concatenation; when a line forks, it means the vector is copied, with each copy directed to different destinations.

non-informative Gaussian prior with zero mean and unit variance is used.

Let $X = \{X^n_{T_n}\}_{n\in\mathcal{I}}$ represent all time series, $\theta = \{\theta_n\}_{n\in\mathcal{I}}$ the GRU parameters, and $\tau = \{\tau_n\}_{n\in\mathcal{I}}$ the precision parameters. Here, X is observed, θ contains learned variables, and τ is defined by τ_{θ_n}.

Given the the likelihood of the observed time series and the priors aforementioned above, the posterior probability is then extracted and is formulated according to Equation (3.2).

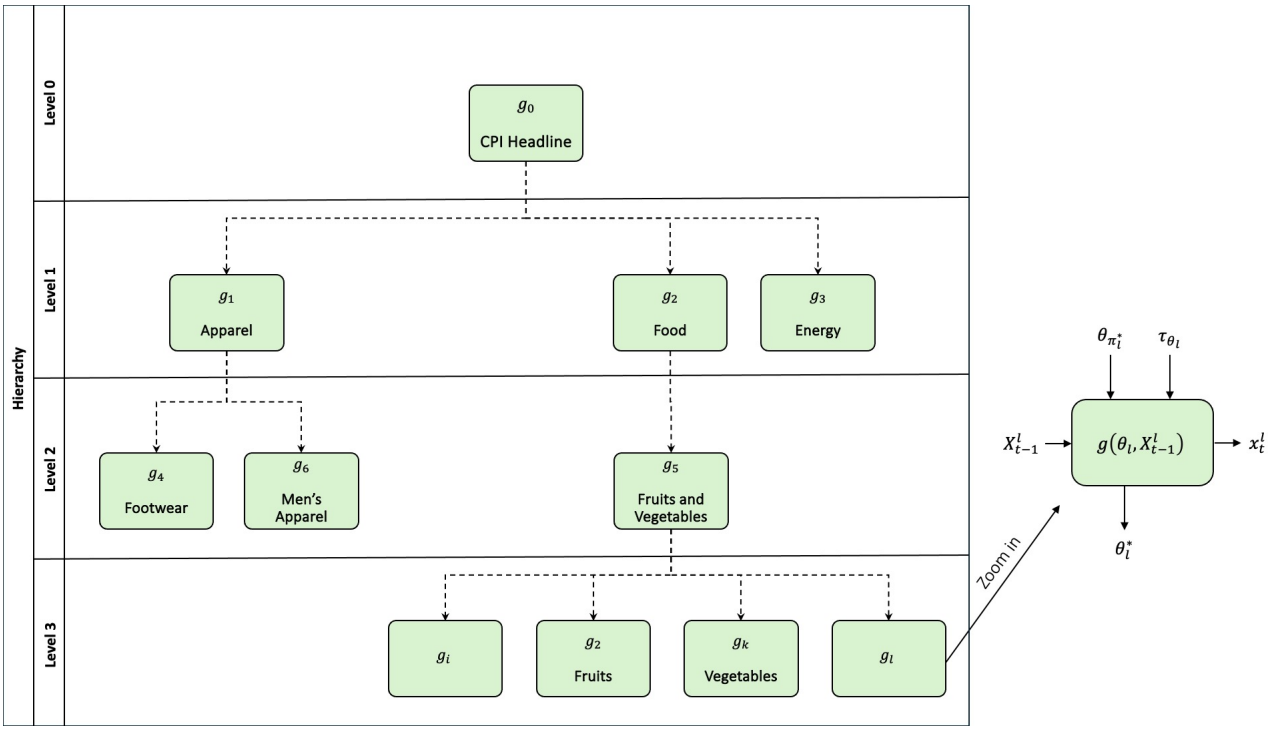
$$p(\theta|X,\tau) = \frac{p(X|\theta,\tau)p(\theta)}{P(X)} \propto \prod_{n\in\mathcal{I}}\prod_{t=1}^{T_n}\mathcal{N}(x^n_t; g(\theta_n, X^n_{t-1}), \tau_n^{-1})\prod_{n\in\mathcal{I}}\mathcal{N}(\theta_n; \theta_{\pi_n}, \tau^{-1}_{\theta_n}\mathbf{I}). \tag{3.2}$$

HRNN optimization follows a *Maximum A-Posteriori* (MAP) approach to find the optimal parameters θ^* by maximizing the posterior probability.

$$\theta^* = \underset{\theta}{\operatorname{argmax}} \log p(\theta|X,\tau). \tag{3.3}$$

The optimization is performed using stochastic gradient ascent on this objective. Figure 3.2 illustrates the HRNN architecture.

Figure 3.2. *Illustration of the HRNN Model.*



3.2 *Bidirectional HRNN Model*

The Bidirectional HRNN (BiHRNN) builds upon the HRNN framework by enabling bidirectional information flow within hierarchical data structures, addressing a critical limitation of its predecessor. While the HRNN model effectively propagates information from parent categories to child categories, improving predictions for lower, more volatile levels, it does not leverage the potential benefits of propagating information in the reverse direction—from child categories back to their parents. Granular-level data often contains unique patterns or anomalies that can inform and refine higher-level predictions. By incorporating bidirectional information flow, BiHRNN enhances the consistency and accuracy of predictions across all levels of the hierarchy.

The motivation for this extension lies in the interconnected nature of hierarchical data, particularly in the context of inflation forecasting. Economic indices at higher aggregation levels, such as national inflation rates, are directly influenced by fluctuations in disaggregated categories, such as specific goods, services, or regional indices. Without accounting for these influences, predictions at higher levels may miss critical insights embedded in lower-level data. By enabling information to flow upward, BiHRNN allows parent-level categories to benefit from the granularity and detail captured at the child level, thereby improving overall forecast accuracy and coherence across the entire hierarchy.

BiHRNN introduces a dual-constraint formulation, which integrates both top-down